



ARTIFICIAL INTELLIGENCE DV-2618/2619

Assignment - 1

Ahmed Mahfooz Ali Khan BTHID- 20030116-8555

Introduction:

This report examines the implementation of three classical search algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), and A*. To solve a pathfinding problem in a 10x20 maze. Each algorithm is evaluated based on its performance in terms of search length, memory usage, path length and time taken. We compare their strengths and weaknesses and determine based on these metrics that which algorithm is best suited and most suitable for this type of maze problem.

Implementation Details:

The maze is represented as a 2D numpy array, where 0 represents open paths and 1 represents walls. The start point is at (0, 1), and the goal is at (9, 18). Each algorithm searches for a path from start to goal using different strategies:

- **BFS:** Explores nodes level by level, visiting all neighboring nodes before moving to the next level, guaranteeing the shortest path in an unweighted graph.
- **DFS:** Explores nodes by going as deep as possible along one branch before backtracking, making it less optimal for shortest paths but useful for traversing entire graphs.
- **A*:** Uses a priority queue (heap) to explore paths based on their total estimated cost ($f_cost = g_cost + h_cost$). It is guided by the Manhattan distance heuristic, making it efficient in finding the shortest path, combining the strengths of BFS with added optimization.

Performance Metrics:

- **Search Length:** Number of nodes explored.
- **Memory Usage:** Memory consumed during the search process.
- **Path length:** Number of steps in the final path from the start to the end.
- **Time Taken:** Time taken to compute the path.

Based on the results of the executed algorithm:

Algorithm	Path Length	Search Length	Memory Usage (bytes)	Time Taken (seconds)
BFS	27	75	4872	0.000169
DFS	27	80	5624	0.000198
A*	27	93	6784	0.000416

Strengths and Weaknesses:

1. Breadth-First Search (BFS)

- **Strengths:**
 - **Shortest Path Guarantee:** BFS guarantees finding the shortest path in an unweighted maze, which is crucial for pathfinding tasks.
 - **Low Search Length:** With a search length of 75, BFS effectively explores fewer nodes than DFS and A*, leading to efficient exploration in many scenarios.
 - **Fast Execution Time:** The lowest execution time (0.000169 seconds) among the three algorithms makes BFS suitable for real-time applications.
- **Weaknesses:**
 - **Wasted Resources on Expansions:** BFS can waste resources by expanding unnecessary nodes, particularly in large or complex mazes.
 - **Time taken for finding path:** Can be slow for larger mazes due to the need to explore all possible paths in the breadth.

2. Depth-First Search (DFS)

- **Strengths:**
 - **Memory Efficiency:** DFS uses slightly more memory (5624 bytes) than BFS but remains relatively efficient. Its space complexity is low compared to BFS, especially in large search spaces.
 - **Simplicity:** The implementation of DFS is straightforward, making it easier to code and understand.
 - **Quick Execution:** With an execution time of 0.000198 seconds, DFS runs quickly, often finishing tasks in a timely manner.
- **Weaknesses:**
 - **Higher Search Length:** DFS had a search length of 80, which is higher than BFS. This can lead to exploring longer paths unnecessarily before finding the optimal route.
 - **No Shortest Path Guarantee:** DFS does not guarantee finding the shortest path, which can be problematic in certain applications.

3. A Search*

- **Strengths:**
 - **Heuristic Efficiency:** A* uses heuristics to guide its search, which can significantly reduce the number of nodes explored compared to uninformed search strategies like BFS and DFS in complex mazes.

- **Optimal Path Finding:** A* guarantees finding the optimal path when using an admissible heuristic, making it suitable for many pathfinding problems.
- **Weaknesses:**
 - **High Memory Usage:** A* has the highest memory consumption (6784 bytes) due to maintaining both the open set and heuristic values. This can be a limiting factor in larger mazes.
 - **Longest Execution Time:** With a time of 0.000416 seconds, A* is slower than both BFS and DFS, primarily due to the overhead of heuristic calculations and managing the priority queue.
 - **Complexity:** The implementation of A* can be more complex than BFS or DFS, which might make it harder to debug or modify.

Insights of Algorithms:

- **Maze size and complexity:** For small, simple mazes, BFS or DFS may be sufficient. For larger, more complex mazes, A* is often the best choice.
- **Need for the shortest path:** If finding the shortest path is crucial, BFS or A* are the better options.
- **Heuristic function availability:** A* requires a heuristic function, which may not be available for all maze problems.
- **Computational resources:** The choice of algorithm may also depend on available computational resources, as A* can be more computationally intensive than BFS or DFS.

In general, A* is often the preferred algorithm for the maze problem due to its ability to find the shortest path efficiently in most cases. However, the specific choice should be made based on the characteristics of the maze and the requirements of the application.

Results:

Based on the performance metrics and the nature of the maze, **BFS** stands out for its efficiency in search length and time taken, while maintaining the same path length as the other algorithms. However, it requires less memory in the given scenario, But sometime could be a drawback in larger or more complex mazes.

DFS, while slightly less efficient in terms of search length and memory usage compared to BFS, still performs admirably, providing a good balance of speed and resource consumption. Its ability to navigate narrow paths effectively makes it a strong choice for similar problems.

A* algorithm despite its longer execution time and memory overhead, remains a powerful algorithm when the shortest path is crucial and heuristic values are relevant for more complex pathfinding scenarios.

In summary, while BFS is optimal for efficiency in this case, **DFS** is recommended for its balanced performance and lower memory requirements in real-world applications where these factors are critical. **A***, although less suited for this specific maze due to its overhead.

References:

1. <https://chatgpt.com/share/66f1e164-66fc-800d-b7fa-4fb7b663e446>
2. Referred lecture notes.